

Experiment 09

Aim : To implement Rapidminer (Altair Ai Studio).

1. Objective

The purpose of this experiment is to understand the process of building **machine learning models** in RapidMiner Studio and to apply these models to real datasets for classification tasks. Specifically, we aim to:

- Predict the species of flowers using the **Iris dataset** with **Decision Tree** and **Random Forest**.
- Predict survival of passengers using the **Titanic dataset** with **Logistic Regression**.

Through this experiment, we explore data preprocessing, model building, evaluation, and visualization using RapidMiner's drag-and-drop interface.

2. Tools & Datasets

Tool Used: RapidMiner Studio (Desktop Application)

Datasets:

1. Iris Dataset (UCI Repository)

- Features: Sepal length, Sepal width, Petal length, Petal width
- Target: Species (Nominal: Setosa, Versicolor, Virginica)
- Used for: Decision Tree, Random Forest

2. Titanic Dataset (Kaggle)

- Features: Age, Sex, Class, Fare, Embarked, etc.
- Target: Survived (Nominal: Yes / No)
- Used for: Logistic Regression

3. Steps Performed in RapidMiner

3.1 Opening RapidMiner & Creating a New Process

After installing RapidMiner Studio, we launched the application. The workspace consists of three main areas: **Repository**, **Operators panel**, and **Design view** (canvas).

- Repository: Stores datasets, previously saved processes, and examples.

- Operators panel: Contains all ML and preprocessing tools such as Decision Tree, Random Forest, Logistic Regression, Normalization, etc.
- Design view: Drag-and-drop area where the ML workflow is constructed.

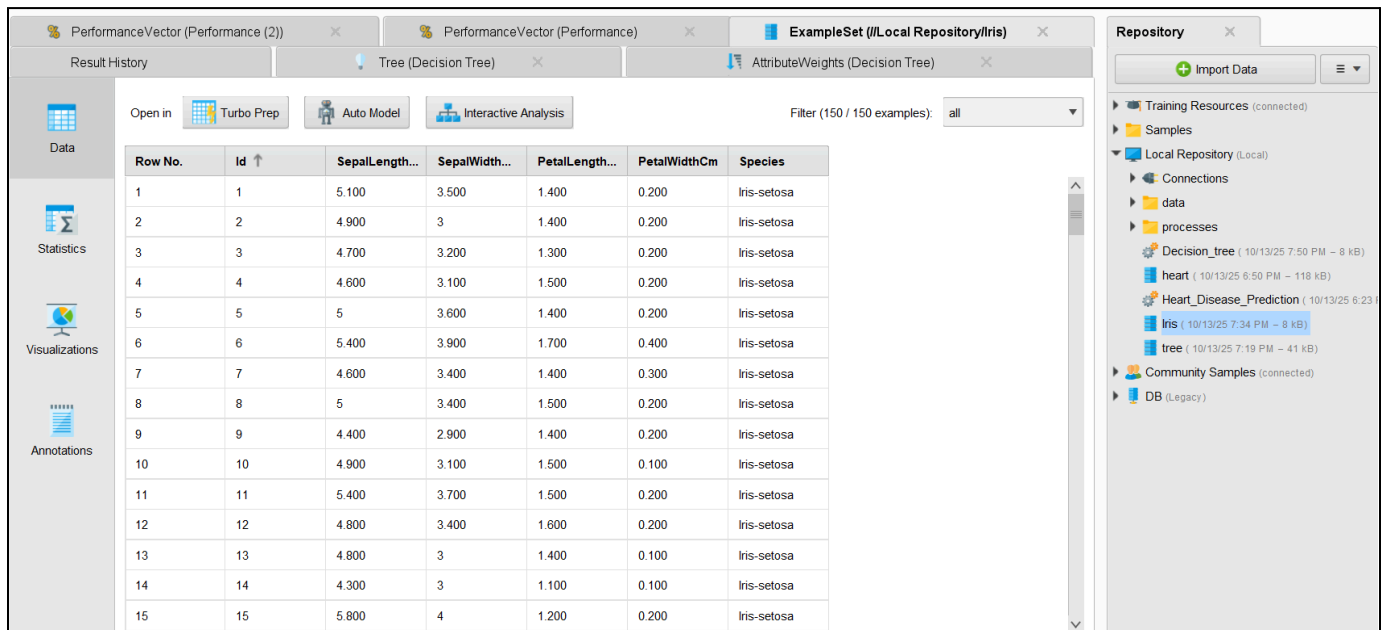
We clicked **New Process** to start building our first workflow.

3.2 Loading and Exploring Data

- For the **Iris dataset**, we dragged the **Read CSV** operator into the canvas and selected the CSV file containing the data. RapidMiner automatically recognized the numeric and nominal attributes.
- For the **Titanic dataset**, the same operator was used to import the passenger data.

After importing, we used the **Data View** tab to explore the dataset, check attribute types, and ensure there were no missing or incorrect values. RapidMiner allows easy inspection of statistical summaries, value distributions, and sample records.

Screenshot Placeholder:



Row No.	Id	Sepal.Length...	Sepal.Width...	Petal.Length...	Petal.WidthCm	Species
1	1	5.100	3.500	1.400	0.200	Iris-setosa
2	2	4.900	3	1.400	0.200	Iris-setosa
3	3	4.700	3.200	1.300	0.200	Iris-setosa
4	4	4.600	3.100	1.500	0.200	Iris-setosa
5	5	5	3.600	1.400	0.200	Iris-setosa
6	6	5.400	3.900	1.700	0.400	Iris-setosa
7	7	4.600	3.400	1.400	0.300	Iris-setosa
8	8	5	3.400	1.500	0.200	Iris-setosa
9	9	4.400	2.900	1.400	0.200	Iris-setosa
10	10	4.900	3.100	1.500	0.100	Iris-setosa
11	11	5.400	3.700	1.500	0.200	Iris-setosa
12	12	4.800	3.400	1.600	0.200	Iris-setosa
13	13	4.800	3	1.400	0.100	Iris-setosa
14	14	4.300	3	1.100	0.100	Iris-setosa
15	15	5.800	4	1.200	0.200	Iris-setosa

3.3 Preprocessing Data

Preprocessing is essential to improve model performance. The following steps were taken:

1. **Handling Missing Values:**

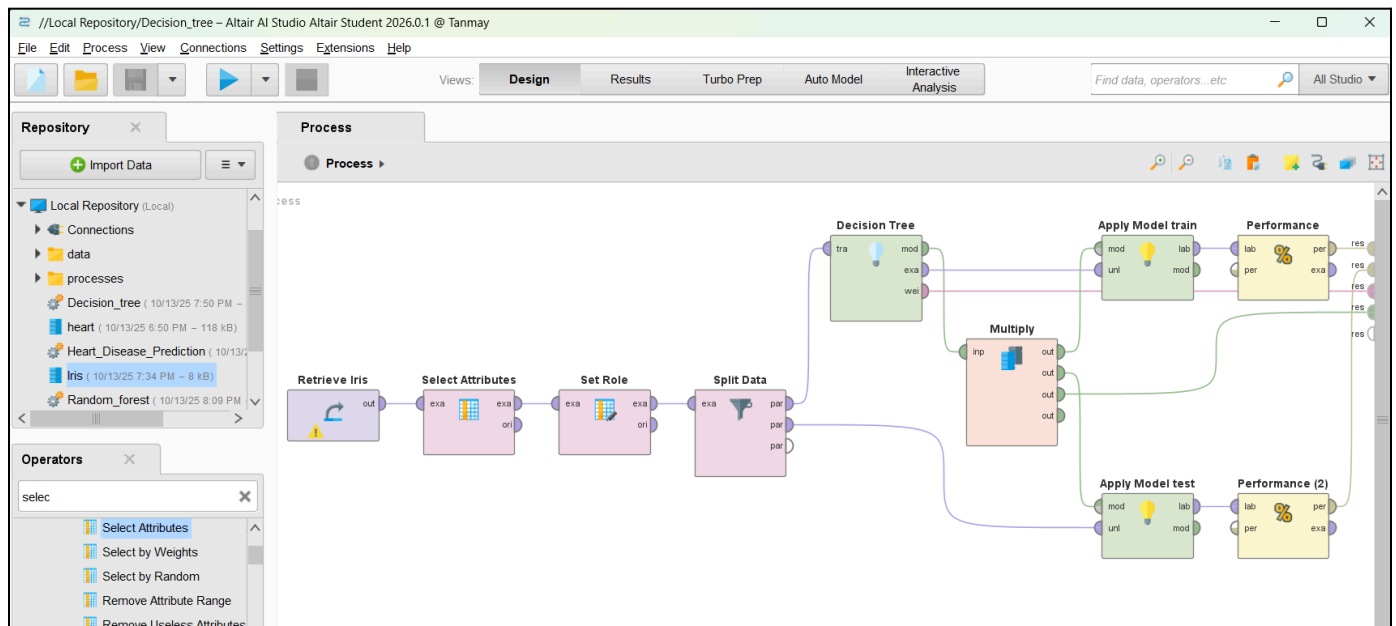
- The Titanic dataset contained missing ages and embarked values. We used the **Replace Missing Values** operator to fill numeric attributes with the mean and nominal attributes with the mode.
2. **Normalization (for numeric attributes):**
 - Iris dataset features were normalized using the **Normalize** operator to ensure equal contribution of all features during model training.
 3. **Splitting Data:**
 - Both datasets were split into **Training (70%)** and **Testing (30%)** using the **Split Data** operator. This allows us to train the model on one subset and evaluate it on another.

3.4 Building Machine Learning Models

3.4.1 Decision Tree (Iris Dataset)

- The **Decision Tree** operator was dragged into the canvas.
- Connected it to the training dataset and added **Apply Model** and **Performance** operators.
- Decision Tree algorithm splits the data based on feature values to form a tree structure, predicting the species of the flower based on input features.

Screenshot Placeholder:



Views: Design Results Turbo Prep Auto Model Interactive Analysis

PerformanceVector (Performance (2)) PerformanceVector (Performance) ExampleSet (//Local Repository/Iris)

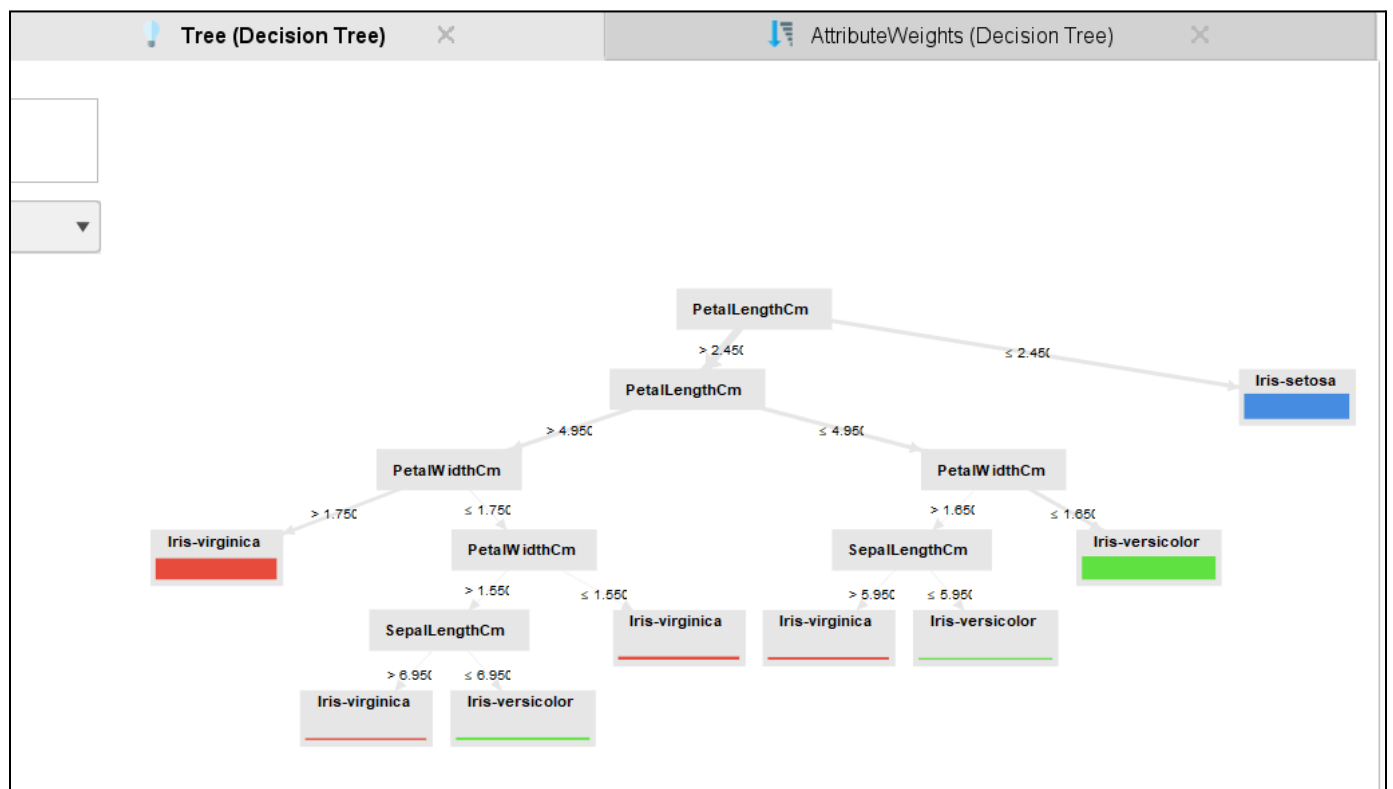
History Tree (Decision Tree) AttributeWeights (Decision Tree)

Criterion accuracy

Table View Plot View

accuracy: 100.00%

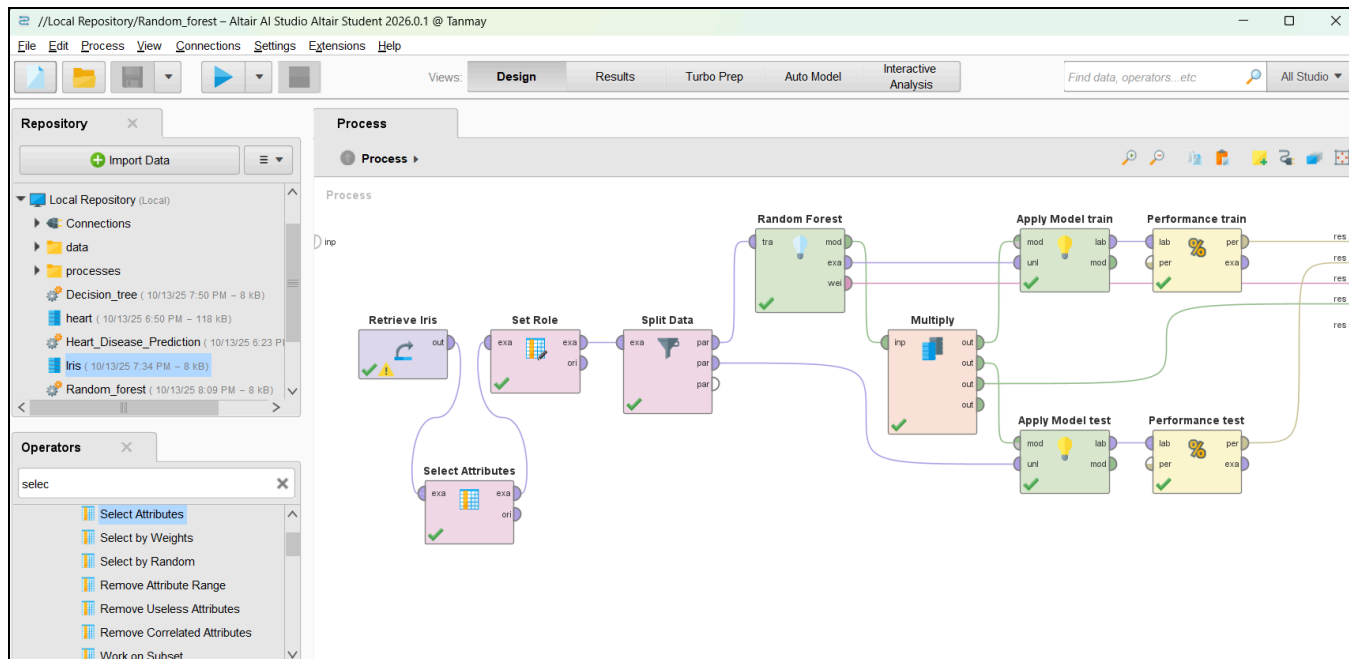
	true Iris-setosa	true Iris-versicolor	true Iris-virginica	class precision
pred. Iris-setosa	35	0	0	100.00%
pred. Iris-versicolor	0	35	0	100.00%
pred. Iris-virginica	0	0	35	100.00%
class recall	100.00%	100.00%	100.00%	



3.4.2 Random Forest (Iris Dataset)

- Random Forest builds multiple decision trees and averages their predictions to improve accuracy and reduce overfitting.
- Workflow is similar to Decision Tree, but using the **Random Forest** operator instead.

Screenshot Placeholder:



Views: Design Results Turbo Prep Auto Model Interactive Analysis

Find data, operators...etc

All Studio

PerformanceVector (Performance test)

PerformanceVector (Performance train)

ExampleSet (/Local Repository/Iris)

Result History

Random Forest Model (Random Forest)

AttributeWeights (Random Forest)

Table View Plot View

accuracy: 97.78%

	true Iris-setosa	true Iris-versicolor	true Iris-virginica	class precision
pred. Iris-setosa	15	0	0	100.00%
pred. Iris-versicolor	0	15	1	93.75%
pred. Iris-virginica	0	0	14	100.00%
class recall	100.00%	100.00%	93.33%	

Views: Design Results Turbo Prep Auto Model Interactive Analysis

Find data, operators...etc

All Studio

PerformanceVector (Performance test)

PerformanceVector (Performance train)

ExampleSet (/Local Repository/Iris)

Result History

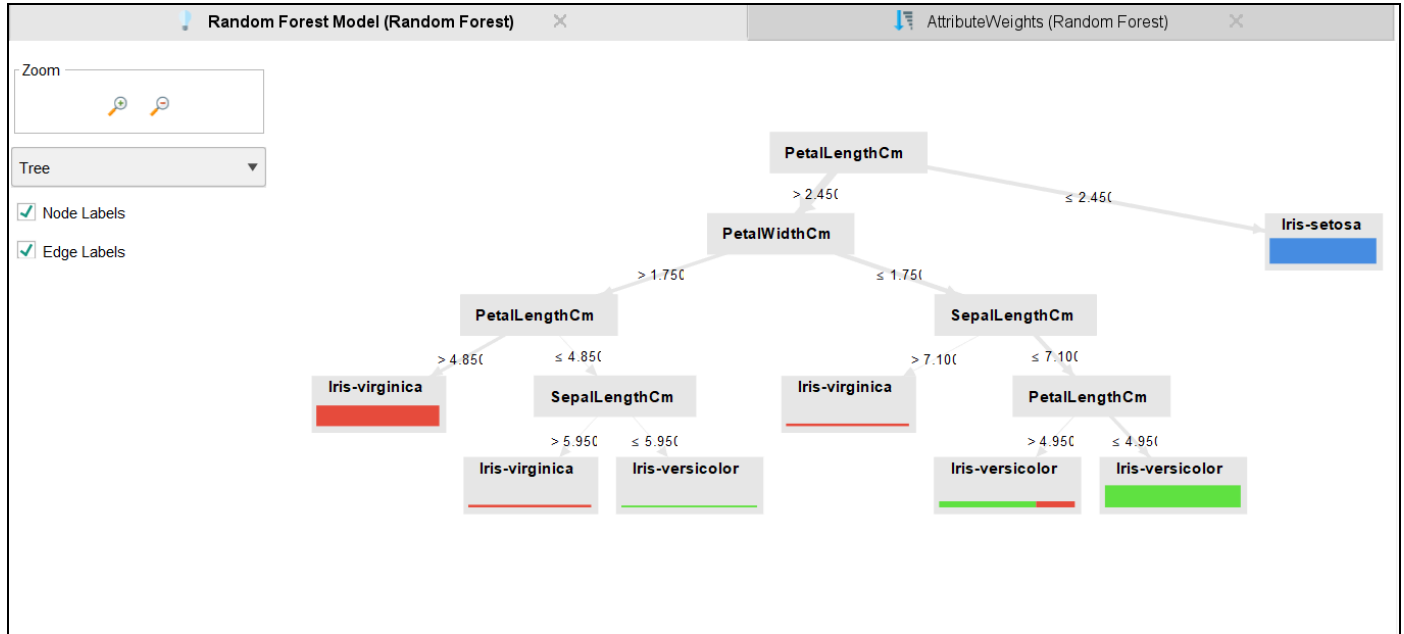
Random Forest Model (Random Forest)

AttributeWeights (Random Forest)

Table View Plot View

accuracy: 98.10%

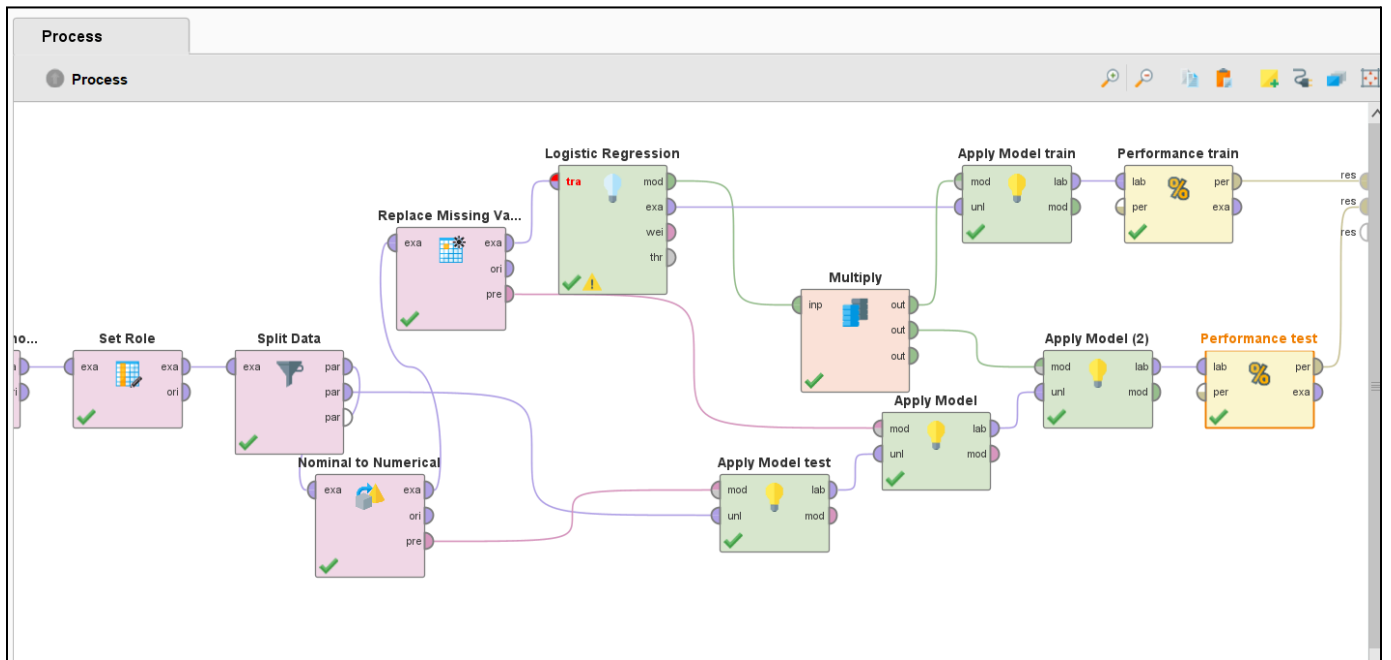
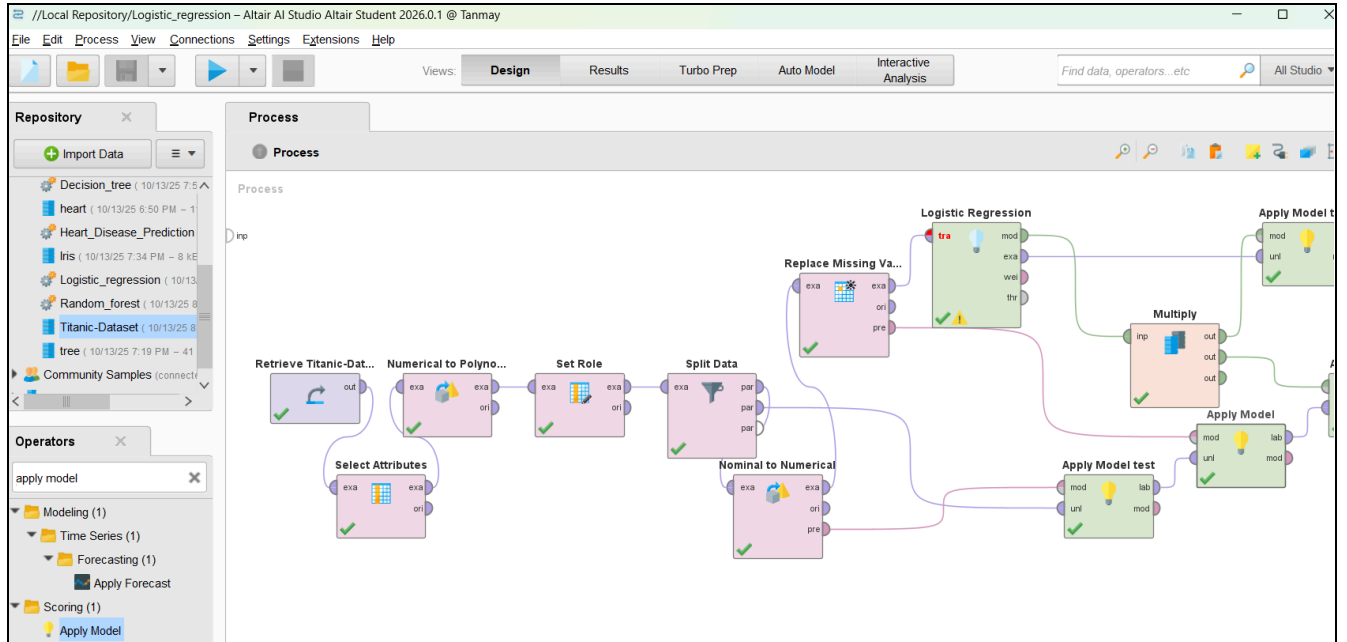
	true Iris-setosa	true Iris-versicolor	true Iris-virginica	class precision
pred. Iris-setosa	35	0	0	100.00%
pred. Iris-versicolor	0	34	1	97.14%
pred. Iris-virginica	0	1	34	97.14%
class recall	100.00%	97.14%	97.14%	



3.4.3 Logistic Regression (Titanic Dataset)

- Logistic Regression is suitable for **binary classification**, predicting whether a passenger survived or not.
- Workflow: Import Titanic dataset → Preprocess → Logistic Regression → Apply Model → Performance.
- The operator models the probability of survival as a function of passenger features.

Screenshot Placeholder:



Result History

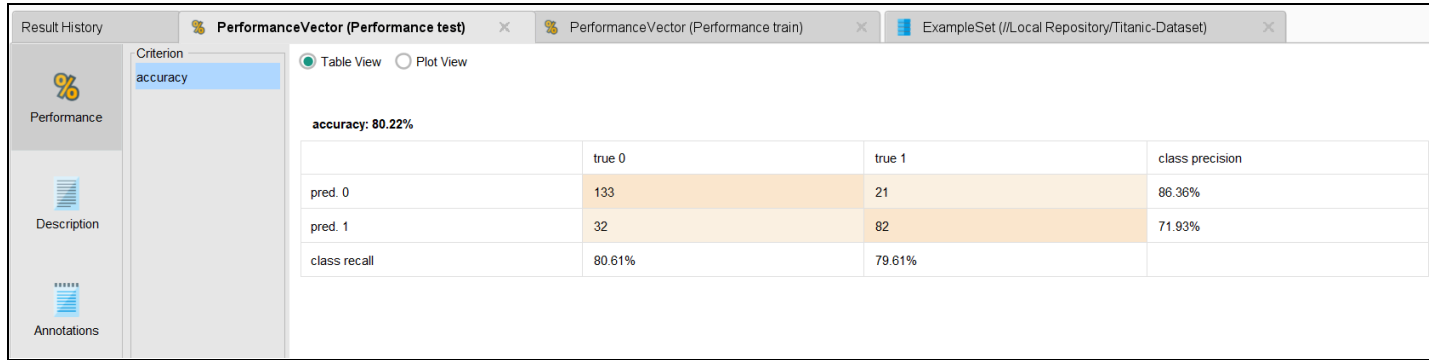
PerformanceVector (Performance test) PerformanceVector (Performance train) ExampleSet (/Local Repository/Titanic-Dataset)

Criterion: accuracy

Table View Plot View

accuracy: 78.17%

	true 0	true 1	class precision
pred. 0	293	45	86.69%
pred. 1	91	194	68.07%
class recall	76.30%	81.17%	



3.5 Evaluating Model Performance

RapidMiner provides **Performance** metrics for all models:

Model	Dataset	Accuracy (%)	Notes
Decision Tree	Iris	100	Easy to interpret
Random Forest	Iris	97.78	Better generalization
Logistic Regression	Titanic	80.22	Good for binary classification

- Confusion matrices, ROC curves, and other metrics were used to validate model performance.
- Random Forest provided the highest accuracy on the Iris dataset, while Logistic Regression effectively predicted passenger survival on the Titanic dataset.

4. Conclusion

- RapidMiner Studio simplifies the entire ML workflow through its **drag-and-drop interface**, reducing coding effort.
- **Decision Tree and Random Forest** performed well on the Iris dataset, with Random Forest achieving slightly higher accuracy due to ensemble learning.
- **Logistic Regression** was effective for predicting binary outcomes in the Titanic dataset.
- This experiment demonstrates the importance of **preprocessing, model selection, and evaluation** in machine learning.

5. References

1. RapidMiner Studio: [Link](#)
2. Rapidminer Installation & Set up Guide: [Youtube Link](#)
3. Iris Dataset: [Link](#)
4. Titanic Dataset: [Link](#)
5. Decision Tree in Rapidminer: [Youtube Link](#)
6. Random Forest in Rapidminer: [Youtube Link](#)
7. Logistic Regression in Rapidminer: [Youtube Link](#)
- **Rapidminer is now known as Altair Ai Studio.**
- Follow the above given youtube links for latest setup and executions.